ACQUIRE.COM LISTING

Morph AI

Your always-on chief of staff. AI-powered calendar and task management — supporting documents for potential buyers.

Morph AI — Technical Architecture

Overview

Morph AI is a production-grade Next.js 14 application. Fully written in TypeScript, uses Prisma ORM for type-safe database access, and is containerized with Docker for portable deployment.

System Architecture

```
┌──────────────────────────────────────────────────────┐
│                Frontend (Next.js 14)                 │
│  ┌──────────┬──────────────────┬────────────────┐    │
│  │  Tasks   │    Calendar      │     Chat       │    │
│  │ Column   │  Month/Week/Day  │  AI Assistant  │    │
│  └──────────┴──────────────────┴────────────────┘    │
│  TanStack Query · Zustand · Tailwind · Radix UI      │
└──────────────────────────────────────────────────────┘
                        ↕ API Routes
┌──────────────────────────────────────────────────────┐
│             Backend (Next.js API Routes)             │
│  /api/chat     → AI conversation, function calling   │
│  /api/events   → Event CRUD                          │
│  /api/tasks    → Task CRUD, reorder, subtasks        │
│  /api/sync     → Google Calendar/Tasks sync          │
│  /api/rag      → Semantic search, reindex            │
│  /api/billing  → Checkout, subscription, manage      │
│  /api/audio    → Transcription (Whisper)             │
│  /api/webhooks → LemonSqueezy payments               │
│  NextAuth.js · Prisma ORM · OpenAI GPT-4o            │
└──────────────────────────────────────────────────────┘
                        ↕
┌──────────────────────────────────────────────────────┐
│         Database (PostgreSQL + pgvector)             │
│  17 models · vector embeddings on Event + Task       │
└──────────────────────────────────────────────────────┘
                        ↕
┌──────────────────────────────────────────────────────┐
│  Google Calendar API · Google Tasks API · OpenAI     │
│  LemonSqueezy · Redis + BullMQ                       │
└──────────────────────────────────────────────────────┘
```

Tech Stack

LAYER	TECHNOLOGY	VERSION
Framework	Next.js (App Router)	14.2

Language	TypeScript	5.4
UI	React + Tailwind CSS + Radix UI	18.3 / 3.4
State	TanStack Query + Zustand	5.28 / 4.5
ORM	Prisma	6.19
Database	PostgreSQL + pgvector	—
Auth	NextAuth.js	4.24
AI	OpenAI SDK	4.104
Queue	BullMQ + Redis (ioredis)	5.4 / 5.3
Billing	LemonSqueezy API	—
Dates	date-fns + chrono-node + rrule	3.3 / 2.9 / 2.8
Testing	Vitest	4.0
Container	Docker (node:20-alpine)	—
Deploy	Railway	—

Database (17 Models)

```
User 1──* Account (OAuth)
User 1──1 Subscriber (billing)
User 1──* Calendar 1──* Event (pgvector embedding)
User 1──* TaskList 1──* Task (pgvector + subtasks)
User 1──* Conversation 1──* Message
User 1──* Document · SyncToken · UsageLog · OpsEvent · BillingEvent
```

Design highlights: pgvector without separate vector DB, soft deletes, sync tokens for incremental Google sync, Serializable isolation for quota enforcement.

AI Pipeline

```
User message → Schedule snapshot → RAG search
→ GPT-4o (function calling) → Preview card
→ User confirms → DB + Google sync
```

INPUT	PROCESSING
Text	GPT-4o with function calling
Image	GPT-4 Vision → structured events
PDF	pdfjs-dist → GPT-4o extraction
Audio	Whisper → GPT-4o processing
Web	Search → GPT-4o extraction

Deployment

- **Docker** multi-stage build (deps → builder → runner)
- **Railway**: 2 services (web + worker) from same image
- Portable to any cloud with Docker support

Code Quality

AREA	IMPLEMENTATION
Types	TypeScript throughout + Prisma generated types
Validation	Zod schemas
Testing	Vitest
Linting	ESLint + @typescript-eslint
Security	RLS, OAuth encryption, HMAC webhook verification